

Reviewer Pack — Minimal Diophantine Degree of Boolean Functions and SAT Clau...

Entry a34756c31b5e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 04:35:16 UTC

Minimal Diophantine Degree of Boolean Functions and SAT Clause Entropy Correlation

Entry ID: a34756c31b5e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 04:35:16 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Geometry) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subsets)

Statement:

For every n -ary boolean function f , the minimal degree of a monic polynomial $p(x_1, \dots, x_n)$ with integer coefficients such that f is represented by p , denoted as $\min_deg(p)$, is linearly correlated with the entropy of the set of clauses S in the SAT instance corresponding to f , i.e., $\min_deg(p) = \Theta(\text{Entropy}(S))$.

Rationale (proposer's reasoning):

Diophantine geometry studies algebraic varieties over the integers and has been applied to various complexity-theoretic problems. The minimal degree of a polynomial representing a boolean function could provide a structural insight into the hardness of SAT instances, potentially leading to new proof techniques.

Taxonomy category: DiophantineGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6dd4fe426cd9e7a2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for a statistically significant number of n -ary boolean functions f , the correlation coefficient between $\min_deg(p)$ and $\text{Entropy}(S)$ is greater than or equal to 0.7 and p -value is less than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal Diophantine degree" AND "Boolean satisfiability" - "monic polynomial representation" IN BOOLEAN Satisfiability - "entropy of clauses" related TO "Diophantine geometry"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.6s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_min_deg(p):
        n = len(p)
        if n == 1:
            return 0
        A = [[Fraction(p[i][j], 2) for j in range(n)] for i in range(n)]
        B = [Fraction(p[i][-1], 2) for i in range(n)]
        for k in range(n):
            max_row = k
            for i in range(k+1, n):
                if abs(A[i][k]) > abs(A[max_row][k]):
                    max_row = i
            A[k], A[max_row] = A[max_row], A[k]
            B[k], B[max_row] = B[max_row], B[k]
            for i in range(k+1, n):
                factor = Fraction(A[i][k], A[k][k])
                for j in range(k, n+1):
                    A[i][j] -= factor * A[k][j]
                B[i] -= factor * B[k]
        return sum(1 for row in A if any(x != 0 for x in row))

    def compute_entropy(clauses):
        total = len(clauses)
        counts = [clauses.count(c) for c in set(clauses)]
```

```

probabilities = [count / total for count in counts]
entropy = -sum(p * math.log2(p) for p in probabilities if p != 0)
return entropy

n_max = 40
instances_tested = 0
metric_value = 0.0
conjecture_holds = True
counterexample = ""

for n in range(5, n_max + 1):
    for _ in range(6): # Ensure at least 30 instances per seed
        f = generate_boolean_function(n)
        clauses = [i for i, val in enumerate(f) if val == 1]
        p = [[0] * (n+1) for _ in range(n)]
        p[0][n] = 1
        for i in range(1, n):
            for j in range(i+1):
                p[i][j] = -p[i-1][j]
                if j < i:
                    p[i][j+1] = p[i-1][j]
        min_deg_p = compute_min_deg(p)
        entropy_S = compute_entropy(clauses)
        metric_value += abs(min_deg_p - entropy_S)
        instances_tested += 1

mean_metric_value = metric_value / instances_tested
support_fraction = instances_tested / (n_max - 4) * 6

if support_fraction >= 0.8:
    return {
        "metric_name": "Correlation between min_deg(p) and Entropy(S)",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    return {
        "metric_name": "Correlation between min_deg(p) and Entropy(S)",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "support_fraction < 0.8"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if not r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
elif support_fraction < 0.2:
    print(f"RESULT: FALSIFIED counterexample='support_fraction < 0.2' first_failing_seed={seed}")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction too low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48a3efcf.py", line 101, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48a3efcf.py", line 68, in run_trial
    min_deg_p = compute_min_deg(p)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48a3efcf.py", line 38, in compute_min_deg
    factor = Fraction(A[i][k], A[k][k])
                  ~~~~~^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the division by zero error in the code to allow for proper computation of min_deg(p) and Entropy(S).

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14095
2	preregistration	ollama_remote	glm4:latest	0	0	9435
3	novelty	ollama_remote	glm4:latest	0	0	8327
4	novelty	ollama_remote	glm4:latest	0	0	9522
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16567
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16504
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12469
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17824
9	judge	ollama_remote	glm4:latest	0	0	21788

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 126531 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a34756c31b5e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a34756c31b5e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a34756c31b5e.tar.gz` (if generated)